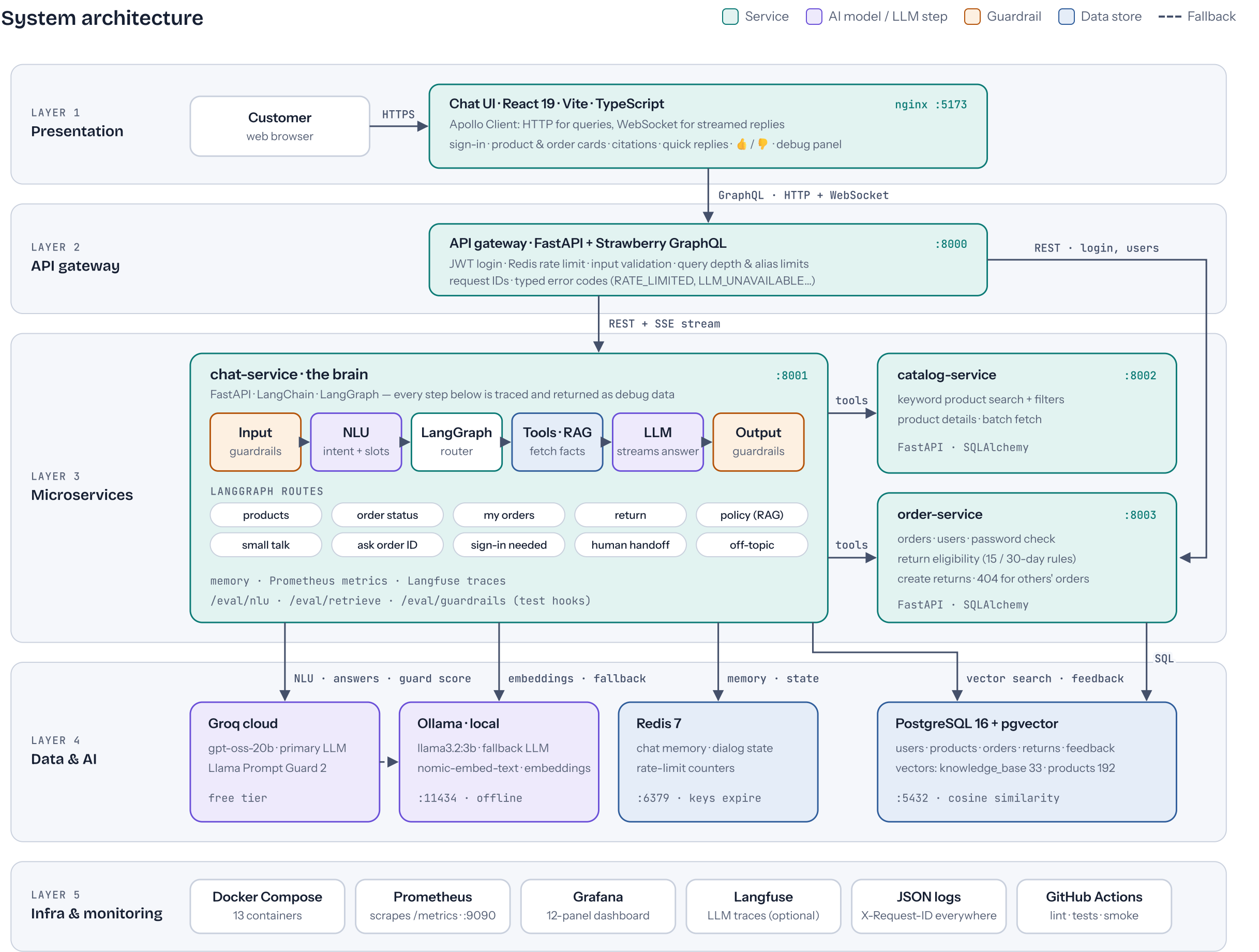


ShopBot architecture

A production-style customer-support chatbot for a fictional store, ShopEase. It answers policy questions with RAG, finds products, tracks orders and starts returns, and is built to be tested end to end with DeepEval, pytest and Playwright.

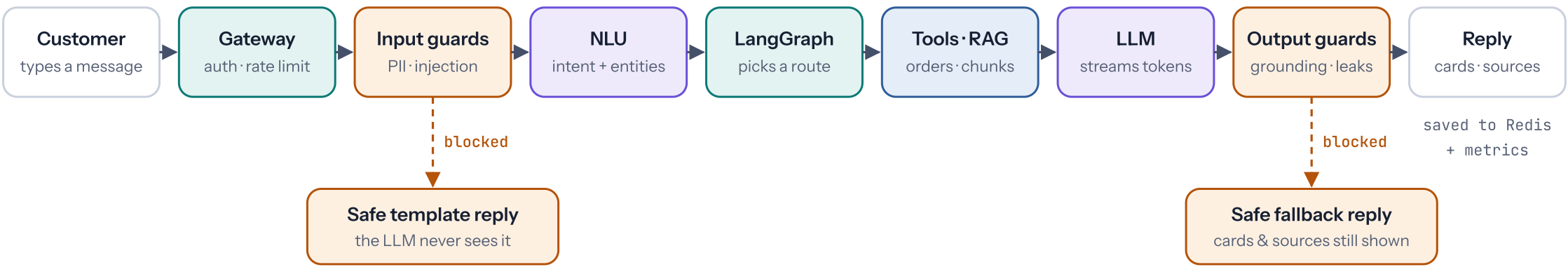
5 layers 3 microservices + GraphQL gateway 225 vectors in pgvector 13 Docker containers 8 guardrail checks \$0 free tools only

System architecture



The UI only ever talks to the gateway. The gateway handles identity and abuse limits, then hands chat to the chat-service, which calls the catalog and order services as LangChain tools and reads facts from PostgreSQL + pgvector before the LLM writes a word. If Groq fails, Ollama takes over (dashed arrow).

How one message flows



Tokens stream to the UI as the LLM writes; the final event carries the guardrail-checked reply, which replaces the streamed text. Sign-in prompts, return confirmations and blocked messages use fixed templates, so they are identical every time.

Tech stack

FRONTEND	API GATEWAY	BACKEND	LLM FRAMEWORK
React 19 · Vite 8 · TypeScript 7 · Apollo Client 3 · graphql-ws · react-markdown Chat UI, streaming, markdown replies · served by nginx	FastAPI · Strawberry GraphQL · PyJWT One endpoint for the UI · auth, rate limits, validation	Python 3.12 · Pydantic v2 · SQLAlchemy 2 (async) · httpx · uv All three microservices	LangChain · LangGraph StateGraph ChatGroq , ChatOllama , .with_fallbacks , .with_structured_output , @tool , prompt templates
MODELS	RAG	DATA	GUARDRAILS
Groq openai/gpt-oss-20b → Ollama Llama3.2:3b Primary on Groq's free tier, local fallback when it fails	nomic-embed-text · pgvector via PgVector Split by heading, then 600 chars with 80 overlap · cosine score ≥ 0.55 and within 0.05 of the best	PostgreSQL 16 + pgvector · Redis 7 Shop data, vectors and feedback in Postgres · sessions and rate limits in Redis	Regex + Luhn check · Llama Prompt Guard 2 PII masking, prompt injection, grounded numbers, prompt-leak checks
OPS			
Prometheus · Grafana · Langfuse · Docker Compose · GitHub Actions · ruff Metrics, dashboards, LLM traces, CI			

Design decisions

One GraphQL gateway

The UI has a single endpoint; auth and abuse limits live in one place.

Layered guardrails

Fast rules plus an ML classifier on the way in, grounding checks on the way out.

Understanding separate from answering

NLU, retrieval and guardrails can each be tested alone through `/eval/*` endpoints.

pgvector, not a separate vector DB

Postgres was already the system of record, and 225 vectors don't need more.

Templates for sensitive paths

Sign-in, return confirmations and refusals don't depend on the LLM: safe, cheap, testable.

Built for testing

Debug data maps to DeepEval fields; offline fake mode, fixed seed data and a fixed "today" make runs repeatable.